2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026 **Week No.: 3** **Program S. No.: 6**

Title	Warehouse Shipment and Logistics Analytics Using NumPy
Problem Statement	A logistics company tracks shipments from 4 warehouses across 7 weeks. Create a 4 x 7 shipment matrix. Perform the following operations: (1) compute average shipments, (2) compute standard deviation, (3) slice the first two warehouses and last three weeks, (4) reshape the matrix into a 7 x 4 matrix, and (5) perform element-wise multiplication with a transportation-cost matrix of the same shape. As an extra practice mini project, develop a Supply Chain Analytics Dashboard that tracks inventory movement, optimizes transportation costs, identifies logistics bottlenecks, generates warehouse performance reports, and supports supply-chain planning.
Objectives	• To create a 4 x 7 warehouse shipment matrix using NumPy. • To store shipment, transportation-cost, and capacity data locally using predefined Python dictionaries. • To calculate the mean and standard deviation of the complete shipment matrix. • To extract the first two warehouses and last three weeks using NumPy slicing. • To reshape the shipment matrix from 4 x 7 into 7 x 4 without changing its elements. • To calculate warehouse-week transportation expenditure using element-wise multiplication. • To track warehouse totals, weekly inventory movement, transportation cost, and capacity utilization. • To identify high-performing warehouses, cost-efficient warehouses, peak weeks, and logistics bottlenecks. • To develop an additional Streamlit dashboard for warehouse reporting and supply-chain planning. • Learning Outcome: Students understand numerical analytics in logistics management.
Dataset URL Link / File	The dataset is stored locally inside the console and dashboard programs using predefined dictionaries. No external CSV file, URL, or database is required. **A. Shipment Matrix (units shipped)**

Warehouse	W1	W2	W3	W4	W5	W6	W7
Warehouse 1	120	135	128	142	150	160	155
Warehouse 2	98	105	110	115	120	118	125
Warehouse 3	140	145	150	155	165	170	175
Warehouse 4	80	85	90	95	100	105	110

B. Transportation Cost Matrix (cost per shipment unit)

Warehouse	W1	W2	W3	W4	W5	W6	W7
Warehouse 1	12	12	13	13	14	14	15
Warehouse 2	10	11	11	12	12	13	13
Warehouse 3	14	14	15	15	16	16	17
Warehouse 4	9	9	10	10	11	11	12

C. Weekly Capacity Dictionary

Warehouse	Weekly Capacity
Warehouse 1	175
Warehouse 2	145
Warehouse 3	180
Warehouse 4	120

Functions Details

Sl. No.	Function	Package	Description of function
1	np.array()	NumPy	Converts dictionary values into two-dimensional NumPy matrices.
2	np.mean()	NumPy	Calculates average shipments for the full matrix or a selected axis.
3	np.std()	NumPy	Measures the variation of shipment values around the mean.
4	Array slicing	NumPy	Extracts the first two warehouses and last three weeks.
5	reshape()	NumPy	Reorganizes the 4 x 7 matrix into a 7 x 4 matrix.
6	np.multiply()	NumPy	Performs element-wise multiplication of shipments and unit costs.
7	np.sum()	NumPy	Calculates warehouse totals, weekly totals, and total transportation cost.
8	np.argmax()	NumPy	Returns the index of the maximum shipment or utilization value.

	9	np.where()	NumPy	Identifies warehouse-week combinations that cross the bottleneck threshold.
	10	pd.DataFrame()	Pandas	Creates labeled warehouse and weekly reports for dashboard display.
	11	st.metric()	Streamlit	Displays important supply-chain key performance indicators.
	12	st.line_chart()/ bar_chart()	Streamlit	Visualizes inventory movement, cost, utilization, and warehouse performance.

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model

Step 1: Import NumPy for numerical operations.
Step 2: Create local dictionaries containing warehouse names, week names, shipment data, transportation costs, and weekly capacities.
Step 3: Convert dictionary values into NumPy arrays.
Step 4: Display the original 4 x 7 shipment and transportation-cost matrices.
Step 5: Calculate the mean of the entire shipment matrix using np.mean().
Step 6: Calculate the standard deviation using np.std().
Step 7: Slice shipments[:2, -3:] to obtain the first two warehouses and last three weeks.
Step 8: Reshape the 4 x 7 shipment matrix into a 7 x 4 matrix.
Step 9: Use np.multiply() to multiply each shipment value by its corresponding transportation cost.
Step 10: Calculate warehouse-wise shipments, week-wise shipments, total costs, average costs, and capacity utilization.
Step 11: Identify the highest-performing warehouse and peak shipment week using np.argmax().
Step 12: Identify logistics bottlenecks where utilization is at least 90 percent of weekly capacity.
Step 13: Generate a warehouse performance report for console output.
Step 14: Create a Streamlit dashboard for inventory movement, transportation-cost analysis, bottleneck reporting, and planning.
Step 15: Execute the console program and dashboard, then capture each required output screenshot.

Program

A. Console Program

```
import numpy as np

# Local dataset stored using predefined dictionaries
shipment_data = {
    "Warehouse 1": [120, 135, 128, 142, 150, 160, 155],
    "Warehouse 2": [98, 105, 110, 115, 120, 118, 125],
    "Warehouse 3": [140, 145, 150, 155, 165, 170, 175],
    "Warehouse 4": [80, 85, 90, 95, 100, 105, 110]
}

transport_cost_data = {
    "Warehouse 1": [12, 12, 13, 13, 14, 14, 15],
    "Warehouse 2": [10, 11, 11, 12, 12, 13, 13],
```

```
    "Warehouse 3": [14, 14, 15, 15, 16, 16, 17],
    "Warehouse 4": [9, 9, 10, 10, 11, 11, 12]
}

capacity_data = {
    "Warehouse 1": 175,
    "Warehouse 2": 145,
    "Warehouse 3": 180,
    "Warehouse 4": 120
}

warehouses = np.array(list(shipment_data.keys()))
weeks = np.array(["Week 1", "Week 2", "Week 3", "Week 4",
                  "Week 5", "Week 6", "Week 7"])
shipments = np.array(list(shipment_data.values()))
transport_cost = np.array(list(transport_cost_data.values()))
capacity = np.array(list(capacity_data.values()))

print("WAREHOUSE SHIPMENT MATRIX")
print(shipments)
print("Shape:", shipments.shape)

print("\nTRANSPORTATION COST MATRIX")
print(transport_cost)
print("Shape:", transport_cost.shape)

average_shipments = np.mean(shipments)
shipment_std = np.std(shipments)

print("\nAVERAGE AND STANDARD DEVIATION")
print("Average shipments:", round(average_shipments, 2))
print("Standard deviation:", round(shipment_std, 2))

sliced_shipments = shipments[:2, -3:]
print("\nFIRST TWO WAREHOUSES AND LAST THREE WEEKS")
print(sliced_shipments)
print("Sliced shape:", sliced_shipments.shape)

reshaped_shipments = shipments.reshape(7, 4)
print("\nRESHAPED SHIPMENT MATRIX")
print(reshaped_shipments)
print("Reshaped shape:", reshaped_shipments.shape)

transportation_expense = np.multiply(shipments, transport_cost)
print("\nELEMENT-WISE TRANSPORTATION EXPENSE")
print(transportation_expense)

warehouse_totals = np.sum(shipments, axis=1)
```

```
weekly_totals = np.sum(shipments, axis=0)
warehouse_costs = np.sum(transportation_expense, axis=1)
average_unit_cost = warehouse_costs / warehouse_totals
utilization = shipments / capacity[:, None] * 100
maximum_utilization = np.max(utilization, axis=1)

best_warehouse_index = np.argmax(warehouse_totals)
peak_week_index = np.argmax(weekly_totals)
efficient_warehouse_index = np.argmin(average_unit_cost)
bottleneck_index = np.argmax(maximum_utilization)
bottleneck_points = np.where(utilization >= 90)

print("\nWAREHOUSE PERFORMANCE REPORT")
for i in range(len(warehouses)):
    print(warehouses[i])
    print("  Total shipments:", warehouse_totals[i])
    print("  Average weekly shipments:", round(np.mean(shipments[i]), 2))
    print("  Transportation expense:", warehouse_costs[i])
    print("  Average cost per shipment:", round(average_unit_cost[i], 2))
    print("  Maximum capacity utilization:",
          round(maximum_utilization[i], 2), "%")

print("\nWEEKLY INVENTORY MOVEMENT")
for i in range(len(weeks)):
    print(weeks[i], ":", weekly_totals[i])

print("\nLOGISTICS BOTTLENECK POINTS")
for warehouse_i, week_i in zip(*bottleneck_points):
    print(warehouses[warehouse_i], weeks[week_i],
          "Utilization:", round(utilization[warehouse_i, week_i], 2), "%")

print("\nSUPPLY CHAIN PLANNING SUMMARY")
print("Highest-performing warehouse:",
      warehouses[best_warehouse_index])
print("Peak shipment week:", weeks[peak_week_index])
print("Most cost-efficient warehouse:",
      warehouses[efficient_warehouse_index])
print("Highest bottleneck risk:", warehouses[bottleneck_index])
print("Total shipments:", np.sum(shipments))
print("Total transportation expense:", np.sum(transportation_expense))
```

B. Extra Practice: Streamlit Supply Chain Analytics Dashboard

```
import numpy as np
import pandas as pd
import streamlit as st

st.set_page_config(
    page_title="Supply Chain Analytics Dashboard",
    layout="wide"
```

```
)

st.title("Warehouse Shipment and Supply Chain Analytics")

# The dashboard uses the same local dictionary dataset
shipment_data = {
   "Warehouse 1": [120, 135, 128, 142, 150, 160, 155],
   "Warehouse 2": [98, 105, 110, 115, 120, 118, 125],
   "Warehouse 3": [140, 145, 150, 155, 165, 170, 175],
   "Warehouse 4": [80, 85, 90, 95, 100, 105, 110]
}

transport_cost_data = {
   "Warehouse 1": [12, 12, 13, 13, 14, 14, 15],
   "Warehouse 2": [10, 11, 11, 12, 12, 13, 13],
   "Warehouse 3": [14, 14, 15, 15, 16, 16, 17],
   "Warehouse 4": [9, 9, 10, 10, 11, 11, 12]
}

capacity_data = {
   "Warehouse 1": 175,
   "Warehouse 2": 145,
   "Warehouse 3": 180,
   "Warehouse 4": 120
}

weeks = ["Week 1", "Week 2", "Week 3", "Week 4",
        "Week 5", "Week 6", "Week 7"]

shipment_df = pd.DataFrame.from_dict(
   shipment_data, orient="index", columns=weeks
)
cost_rate_df = pd.DataFrame.from_dict(
   transport_cost_data, orient="index", columns=weeks
)
expense_df = shipment_df * cost_rate_df
capacity_series = pd.Series(capacity_data)
utilization_df = shipment_df.div(capacity_series, axis=0) * 100

warehouse_totals = shipment_df.sum(axis=1)
weekly_totals = shipment_df.sum(axis=0)
warehouse_expense = expense_df.sum(axis=1)
average_cost = warehouse_expense / warehouse_totals
maximum_utilization = utilization_df.max(axis=1)

performance_report = pd.DataFrame({
   "Total Shipments": warehouse_totals,
   "Average Weekly Shipments": shipment_df.mean(axis=1).round(2),
```

```
    "Transportation Expense": warehouse_expense,
    "Average Cost per Shipment": average_cost.round(2),
    "Maximum Utilization (%)": maximum_utilization.round(2)
})

best_warehouse = warehouse_totals.idxmax()
peak_week = weekly_totals.idxmax()
cost_efficient = average_cost.idxmin()
bottleneck = maximum_utilization.idxmax()

selected_warehouse = st.sidebar.selectbox(
    "Select warehouse", shipment_df.index
)
threshold = st.sidebar.slider(
    "Bottleneck threshold (%)", 70, 100, 90
)

c1, c2, c3, c4 = st.columns(4)
c1.metric("Total Shipments", f"{int(shipment_df.values.sum()):,}")
c2.metric("Top Warehouse", best_warehouse)
c3.metric("Peak Week", peak_week)
c4.metric("Bottleneck Risk", bottleneck)

movement_tab, cost_tab, bottleneck_tab, report_tab, planning_tab =
st.tabs([
    "Inventory Movement", "Transportation Costs",
    "Logistics Bottlenecks", "Warehouse Reports",
    "Supply Chain Planning"
])

with movement_tab:
    st.subheader("Weekly Inventory Movement")
    st.line_chart(shipment_df.T)
    st.bar_chart(weekly_totals)
    st.dataframe(shipment_df, use_container_width=True)

with cost_tab:
    st.subheader("Transportation Expense by Warehouse")
    st.bar_chart(warehouse_expense)
    st.write("Most cost-efficient warehouse:", cost_efficient)
    st.dataframe(expense_df, use_container_width=True)

with bottleneck_tab:
    st.subheader("Capacity Utilization")
    st.line_chart(utilization_df.T)
    bottleneck_table = utilization_df[utilization_df >= threshold]
    st.dataframe(bottleneck_table, use_container_width=True)
    if bottleneck_table.notna().any().any():
```

```
        st.warning("High-utilization warehouse-week combinations
detected.")
    else:
        st.success("No bottlenecks detected at the selected threshold.")

with report_tab:
    st.subheader("Warehouse Performance Report")
    st.dataframe(performance_report, use_container_width=True)
    st.bar_chart(performance_report[["Total Shipments"]])

with planning_tab:
    st.subheader("Supply Chain Planning Summary")
    st.write("Selected warehouse:", selected_warehouse)
    st.write("Highest-performing warehouse:", best_warehouse)
    st.write("Peak shipment week:", peak_week)
    st.write("Most cost-efficient warehouse:", cost_efficient)
    st.write("Highest bottleneck risk:", bottleneck)
    st.write(
        "Planning action: Review vehicle allocation, dispatch frequency, "
        "capacity, and route cost for high-utilization warehouses."
    )

# Run using: python -m streamlit run dashboard.py
```

Output Snapshots With Explanation/ Observations

Screenshot 1: Original Shipment and Transportation-Cost Matrices

```
WAREHOUSE SHIPMENT MATRIX
[[120 135 128 142 150 160 155]
 [ 98 105 110 115 120 118 125]
 [140 145 150 155 165 170 175]
 [ 80  85  90  95 100 105 110]]
Shape: (4, 7)

TRANSPORTATION COST MATRIX
[[12 12 13 13 14 14 15]
 [10 11 11 12 12 13 13]
 [14 14 15 15 16 16 17]
 [ 9  9 10 10 11 11 12]]
Shape: (4, 7)
```

Figure 1.1 Original Shipment & Transportation Cost Matrix

Explanation/Observation: The local dictionary data was converted into two NumPy matrices of shape (4, 7), representing four warehouses across seven weeks.

Screenshot 2: Average Shipment and Standard Deviation

```
AVERAGE AND STANDARD DEVIATION
Average shipments: 126.64
Standard deviation: 26.66
```

Figure 1.2 Average & Standard Deviation

Explanation/Observation: The mean represents the average shipment volume across all warehouse-week observations, while the standard deviation measures shipment variation.

Screenshot 3: Sliced Shipment Matrix

```
FIRST TWO WAREHOUSES AND LAST THREE WEEKS
[[150 160 155]
 [120 118 125]]
Sliced shape: (2, 3)
```

Figure 1.3 Sliced Shipment Matrix

Explanation/Observation: The expression shipments[:2, -3:] selected the first two warehouses and the last three weeks, producing a matrix of shape (2, 3).

Screenshot 4: Reshaped 7 x 4 Shipment Matrix

```
RESHAPED SHIPMENT MATRIX
[[120 135 128 142]
 [150 160 155  98]
 [105 110 115 120]
 [118 125 140 145]
 [150 155 165 170]
 [175  80  85  90]
 [ 95 100 105 110]]
Reshaped shape: (7, 4)
```

Figure 1.4 Reshaped Matrix

Explanation/Observation: The reshape operation reorganized all 28 shipment values into seven rows and four columns without adding or removing data.

Screenshot 5: Element-wise Transportation Expense Matrix

```
ELEMENT-WISE TRANSPORTATION EXPENSE
[[1440 1620 1664 1846 2100 2240 2325]
 [ 980 1155 1210 1380 1440 1534 1625]
 [1960 2030 2250 2325 2640 2720 2975]
 [ 720  765  900  950 1100 1155 1320]]
```

Figure 1.5 Element Wise Matrix

Explanation/Observation: Each shipment quantity was multiplied by its corresponding transportation cost to estimate warehouse-week transportation expenditure.

Screenshot 6: Warehouse Performance and Bottleneck Report

```
WAREHOUSE PERFORMANCE REPORT
Warehouse 1
  Total shipments: 990
  Average weekly shipments: 141.43
  Transportation expense: 13235
  Average cost per shipment: 13.37
  Maximum capacity utilization: 91.43 %
Warehouse 2
  Total shipments: 791
  Average weekly shipments: 113.0
  Transportation expense: 9324
  Average cost per shipment: 11.79
  Maximum capacity utilization: 86.21 %
Warehouse 3
  Total shipments: 1100
  Average weekly shipments: 157.14
  Transportation expense: 16900
  Average cost per shipment: 15.36
  Maximum capacity utilization: 97.22 %
Warehouse 4
  Total shipments: 665
  Average weekly shipments: 95.0
  Transportation expense: 6910
  Average cost per shipment: 10.39
  Maximum capacity utilization: 91.67 %
```

Figure 1.6 Warehouse Performance Report

Explanation/Observation: The console report compared total shipments, average shipments, transportation expense, cost efficiency, and maximum capacity utilization for every warehouse.

Screenshot 7: Cost, Bottleneck, and Planning Views

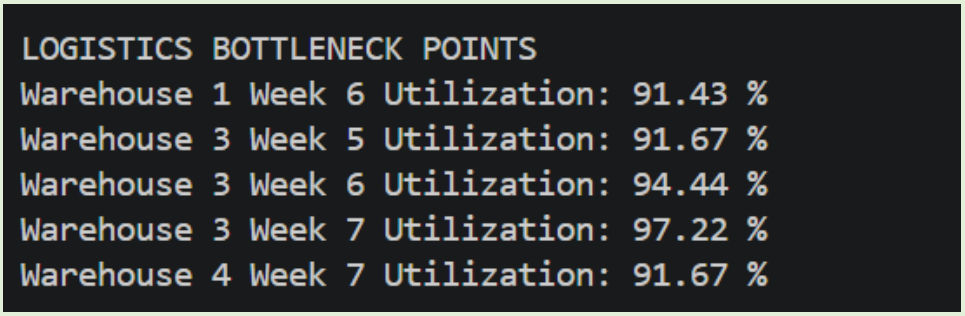

Figure 1.7 Logistics Bottleneck Points

Explanation/Observation: We visualized transportation expenses and capacity utilization, identified high-utilization periods, and generated planning recommendations.

Conclusions

The Warehouse Shipment and Logistics Analytics exercise was completed successfully using NumPy. Shipment, transportation-cost, and capacity data were stored locally using predefined dictionaries and converted into structured arrays. The mean and standard deviation of the shipment matrix were calculated, the first two warehouses and last three weeks were extracted through slicing, and the matrix was reshaped from 4 x 7 to 7 x 4. Element-wise multiplication was used to calculate transportation expenditure. Additional warehouse-wise and week-wise calculations identified the highest-performing warehouse, peak shipment week, most cost-efficient warehouse, and high-utilization bottlenecks.

References

1. NumPy documentation for array creation, descriptive statistics, slicing, reshaping, aggregation, and element-wise operations.
2. Pandas documentation for DataFrame creation and warehouse performance reporting.
3. Python Program Link (https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%203/EXERCISE_PROGRAM.py)